

PRACTICAL-01

```
#include <iostream>
```

```
#include <vector>
```

```
#include <cstdlib>
```

```
#include <ctime>
```

```
#include <chrono>
```

```
using namespace std;
```

```
using namespace std::chrono;
```

```
----- Bubble Sort -----
```

```
void bubbleSort(vector<int> &arr)
```

```
{
```

```
    int size = arr.size();
```

```
    for (int pass = 0; pass < size - 1; pass++)
```

```
    {
```

```
        for (int i = 0; i < size - pass - 1; i++)
```

```
        {
```

```
            if (arr[i] > arr[i + 1])
```

```
            {
```

```
                swap(arr[i], arr[i + 1]);
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
----- Selection Sort -----
```

```
void selectionSort(vector<int> &arr)
```

```
{
```

```
    int size = arr.size();
```

```
    for (int i = 0; i < size - 1; i++)
```

PRACTICAL-01

```
{  
    int minIndex = i;  
  
    for (int j = i + 1; j < size; j++)  
    {  
        if (arr[j] < arr[minIndex])  
        {  
            minIndex = j;  
        }  
    }  
  
    swap(arr[i], arr[minIndex]);  
}  
}
```

----- Insertion Sort -----

```
void insertionSort(vector<int> &arr)  
{  
    int size = arr.size();  
  
    for (int i = 1; i < size; i++)  
    {  
        int current = arr[i];  
        int j = i - 1;  
  
        while (j >= 0 && arr[j] > current)  
        {  
            arr[j + 1] = arr[j];  
            j--;  
        }  
    }  
}
```

PRACTICAL-01

```
    arr[j + 1] = current;
}
}
```

----- Merge Sort -----

```
void merge(vector<int> &arr, int left, int mid, int right)
```

```
{
    int leftSize = mid - left + 1;
    int rightSize = right - mid;

    vector<int> leftArray(leftSize);
    vector<int> rightArray(rightSize);
```

```
    for (int i = 0; i < leftSize; i++)
        leftArray[i] = arr[left + i];
```

```
    for (int i = 0; i < rightSize; i++)
        rightArray[i] = arr[mid + 1 + i];
```

```
    int i = 0, j = 0, k = left;
```

```
    while (i < leftSize && j < rightSize)
```

```
{
    if (leftArray[i] <= rightArray[j])
        arr[k++] = leftArray[i++];
    else
        arr[k++] = rightArray[j++];
}
```

```
    while (i < leftSize)
```

```
        arr[k++] = leftArray[i++];
```

PRACTICAL-01

```
while (j < rightSize)
    arr[k++] = rightArray[j++];
}
```

```
void mergeSort(vector<int> &arr, int left, int right)
{
    if (left < right)
    {
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}
```

----- Quick Sort -----

```
int partition(vector<int> &arr, int low, int high)
{
    int pivot = arr[high];
    int smallerIndex = low - 1;

    for (int i = low; i < high; i++)
    {
        if (arr[i] < pivot)
        {
            smallerIndex++;
            swap(arr[smallerIndex], arr[i]);
        }
    }
}
```

PRACTICAL-01

```
}
```

```
swap(arr[smallerIndex + 1], arr[high]);
```

```
return smallerIndex + 1;
```

```
}
```

```
void quickSort(vector<int> &arr, int low, int high)
```

```
{
```

```
    if (low < high)
```

```
    {
```

```
        int pivotIndex = partition(arr, low, high);
```

```
        quickSort(arr, low, pivotIndex - 1);
```

```
        quickSort(arr, pivotIndex + 1, high);
```

```
    }
```

```
}
```

```
----- Main Function -----
```

```
int main()
```

```
{
```

```
    const int SIZE = 100;
```

```
    vector<int> originalArray(SIZE);
```

```
    vector<int> tempArray;
```

```
    srand(time(0));
```

```
    for (int i = 0; i < SIZE; i++)
```

```
    {
```

```
        originalArray[i] = rand() % 1000;
```

```
    }
```

PRACTICAL-01

```
cout << "Number of Elements = " << SIZE << "\n\n";
```

```
auto start = high_resolution_clock::now();
```

```
tempArray = originalArray;
```

```
bubbleSort(tempArray);
```

```
auto stop = high_resolution_clock::now();
```

```
cout << "Bubble Sort Time : "
```

```
    << duration_cast<microseconds>(stop - start).count()
```

```
    << " microseconds\n";
```

```
start = high_resolution_clock::now();
```

```
tempArray = originalArray;
```

```
selectionSort(tempArray);
```

```
stop = high_resolution_clock::now();
```

```
cout << "Selection Sort Time : "
```

```
    << duration_cast<microseconds>(stop - start).count()
```

```
    << " microseconds\n";
```

```
start = high_resolution_clock::now();
```

```
tempArray = originalArray;
```

```
insertionSort(tempArray);
```

```
stop = high_resolution_clock::now();
```

```
cout << "Insertion Sort Time : "
```

```
    << duration_cast<microseconds>(stop - start).count()
```

```
    << " microseconds\n";
```

```
start = high_resolution_clock::now();
```

```
tempArray = originalArray;
```

```
mergeSort(tempArray, 0, SIZE - 1);
```

```
stop = high_resolution_clock::now();
```

PRACTICAL-01

```
cout << "Merge Sort Time    : "  
    << duration_cast<microseconds>(stop - start).count()  
    << " microseconds\n";
```

```
start = high_resolution_clock::now();  
tempArray = originalArray;  
quickSort(tempArray, 0, SIZE - 1);  
stop = high_resolution_clock::now();  
cout << "Quick Sort Time    : "  
    << duration_cast<microseconds>(stop - start).count()  
    << " microseconds\n";
```

```
return 0;
```

```
}
```